あなたはGodot 4.6.1 (.NET/C#) プロジェクトの開発アシスタントです。

【環境】

- プロジェクトフォルダ: D:\code_workspace\generation-roguelite\generation-roguelite
- Godot: D:\copilot_script\Godot_v4.6.1-stable_mono_win64\Godot_v4.6.1-stable_mono_win64_console.exe
- adb: C:\Users\mikami\AppData\Local\Android\Sdk\platform-tools\adb.exe
- パッケージ名: com.example.generationroguelite

【ルール】

- ファイル書き込みはBOMなしUTF-8: [System.IO.File]::WriteAllText($path, $content, [System.Text.UTF8Encoding]::new($false))
- Set-Content は使用禁止
- C#ファイル名とクラス名は一致させる
- 既存のScripts/Core/やScenes/Main.tscnには一切触れない
- 変更対象はScripts/Sample/とScenes/Sample/のみ

【アセット配置（既にコピー済み）】 res://Assets/Sprites/ — female_idle/run/move/guard/hurt/death.png, navigator.png res://Assets/UI/ — ui_*.webp, navi_bubble_thought.webp, fx_combo.webp res://Assets/Effects/ — fx_guard_shield/hit/just_avoid/particle/zone.webp res://Assets/Objects/ — obstacle_right/top/bottom.webp res://Assets/Backgrounds/ — sky_bg1～4.webp, ground_far/mid/near.webp, midground_*.webp

【全スプライトシート共通】 1024×1024、3×3グリッド、9フレーム、1フレーム341×341 Prompt 1 / 6 — 障害物システム全面書き直し（軌道貫通型＋4方向） Scripts/Sample/GridSampleManager.cs の障害物システムを全面的に書き直してください。 以下の仕様に従い、既存のObstacleData構造体とWave関連コードを完全に置き換えます。

【障害物の根本設計変更：軌道貫通型】 障害物は瞬間着弾ではなく、マスを1つずつ順に通過する。 通過中の全マスに当たり判定がある。

飛来方向は4方向:

- Right: 同じ行をcol2→col1→col0へ横断
- Left: 同じ行をcol0→col1→col2へ横断
- Top: 同じ列をrow0→row1→row2へ縦断
- Down: 同じ列をrow2→row1→row0へ縦断（下から上へ）

【障害物データ構造】

enum ObstacleDirection { Right, Left, Top, Down }

struct ObstacleEntry { public ObstacleDirection Direction; public int Lane; // 行番号(Right/Leftの場合) or 列番号(Top/Downの場合) public float SpawnTime; // この障害物がWave開始から何秒後に出現するか public float SpeedPerCell; // 1マス通過にかかる秒数（0.2～0.5秒） public int OrderNumber; // 予告番号 }

障害物の状態管理: struct ObstacleState { public ObstacleEntry Entry; public float ElapsedSinceSpawn; // 出現からの経過時間 public int CurrentCellIndex; // 現在通過中のセルインデックス（0,1,2） public bool IsActive; // まだ画面内にいるか public bool[] HitApplied; // 各セルでの当たり判定済みフラグ[3]

```
    // ビジュアル
    public TextureRect Visual;       // 障害物画像
```

}

【障害物の移動ロジック】

各障害物は3マスを順に通過する（9マスグリッドは3列×3行なので）。

Right方向の場合: 通過マス順: (col=2,row=Lane) → (col=1,row=Lane) → (col=0,row=Lane) Left方向の場合: 通過マス順: (col=0,row=Lane) → (col=1,row=Lane) → (col=2,row=Lane) Top方向の場合: 通過マス順: (col=Lane,row=0) → (col=Lane,row=1) → (col=Lane,row=2) Down方向の場合: 通過マス順: (col=Lane,row=2) → (col=Lane,row=1) → (col=Lane,row=0)

経過時間からどのマスにいるかを算出: int cellIndex = (int)(ElapsedSinceSpawn / SpeedPerCell); cellIndexが0〜2の間はマス通過中、3以上で画面外消滅。

障害物ビジュアルの位置: cellIndex内での進捗率 = (ElapsedSinceSpawn % SpeedPerCell) / SpeedPerCell 2つのマスの中心座標間を線形補間して障害物のPositionを更新 ※ cellIndex=0の場合は画面外→最初のマスへの補間 ※ cellIndex=2で進捗率が1.0を超えたら最後のマス→画面外へ

【当たり判定】

_Process毎フレームで全アクティブ障害物をチェック: 障害物が現在通過中のマス(col, row)を算出 プレイヤーが同じ(col, row)にいる場合: HitApplied[cellIndex]がfalseなら: 被弾処理（ライフ減少、hurtアニメ等） HitApplied[cellIndex] = true ※ 同じ障害物の同じマスでは1回しか被弾しない

ガード中の場合: ダメージ80%カット ゾーン中の場合: ダメージなし＋障害物破壊演出

【予告システム】

障害物出現のWarningLeadTime=1.5秒前に予告を表示。 予告内容:

- 障害物が通過する全3マスに薄い赤ハイライト
- 飛来方向を示す矢印（Direction別にui_warning_icon.webpを回転表示）
- 進入マス（最初に通過するマス）に予告番号を表示
- 予告マスの赤は時間経過で徐々に濃くなる
- 予告時にナビが方向を伝える（初回のみ）

予告のビジュアル: 各通過マスにColorRectを重ねる（色: Color(1, 0.2, 0.2, alpha)） alphaは 0.1 → 0.4 に時間経過で増加 予告番号はLabelで進入マスに表示（フォント36px）

【障害物ビジュアル】

方向別テクスチャ: Right: obstacle_right.webp Left: obstacle_right.webp を水平反転（FlipH=true） Top: obstacle_top.webp Down: obstacle_bottom.webp サイズ: 96×96のTextureRect

【Wave定義の更新】

struct WaveDef { public ObstacleEntry[] Obstacles; }

サンプルWave（3Wave構成、青年期想定=右＋下の2方向）:

Wave1（チュートリアル的）: 障害物3個、全て右から、間隔2秒、SpeedPerCell=0.4秒 1: Right, Lane=0（上段）, SpawnTime=2.0 2: Right, Lane=1（中段）, SpawnTime=4.0 3: Right, Lane=2（下段）, SpawnTime=6.0

Wave2（2方向導入）: 障害物5個、右＋下、間隔1.5秒、SpeedPerCell=0.35秒 1: Right, Lane=1, SpawnTime=2.0 2: Top, Lane=0, SpawnTime=3.5 3: Right, Lane=0, SpawnTime=5.0 4: Top, Lane=2, SpawnTime=6.5 5: Right, Lane=2, SpawnTime=7.5

Wave3（同時飛来あり・緩急あり）: 障害物8個、右＋下、SpeedPerCell=0.2〜0.45秒（緩急） 1: Right, Lane=0, SpawnTime=2.0, Speed=0.35 2: Right, Lane=2, SpawnTime=2.5, Speed=0.35（同時に近い） 3: Top, Lane=1, SpawnTime=4.0, Speed=0.25（速い） 4: Right, Lane=1, SpawnTime=5.0, Speed=0.45（遅い） 5: Top, Lane=0, SpawnTime=6.0, Speed=0.2（高速） 6: Right, Lane=0, SpawnTime=7.0, Speed=0.3 7: Top, Lane=2, SpawnTime=7.5, Speed=0.3（同時に近い） 8: Right, Lane=1, SpawnTime=9.0, Speed=0.2（高速フィニッシュ）

Wave終了判定: 全障害物がIsActive=falseになったら終了。

【既存コードからの変更点】

- ObstacleData構造体 → ObstacleEntry + ObstacleState に置換
- WaveDef構造体を上記に置換
- TickWave() を全面書き直し
- ShowWarning / ResolveObstacle → 新しいロジックに置換
- 障害物のColorRect → TextureRectに変更

他のシステム（移動、ガード、コンボ、ゾーン、HUD、ナビ等）は変更しないでください。 グリッド座標計算の定数も変更しないでください。

dotnet build でエラーがないことを確認してください。 Prompt 2 / 6 — ジャスト回避（滞在型）実装 GridSampleManager.cs にジャスト回避（滞在型）の判定ロジックを実装してください。

【ジャスト回避の定義】 以下の条件を全て満たした場合にジャスト回避が成立する:

1. 障害物が自分のいるマスに到達する前に、そのマスに0.3秒以上連続して滞在していた
2. 障害物がそのマスに到達する0.4秒前〜到達の瞬間の間にマスから移動を完了した

つまり「あえて危険な場所に居座り、ギリギリで逃げた」ことを検出する。

【実装に必要なデータ】

- float _stayTimer: 現在のマスに連続して滞在している時間（移動完了のたびに0リセット）
- _Process内で _stayTimer += dt（移動中はカウントしない）
- 移動完了時（OnMoveComplete）に以下を記録: int _lastLeftCol, _lastLeftRow: 直前まで居たマス座標 float _lastMoveTime: 移動完了時刻（Time.GetTicksMsec()等） float _stayTimeAtLastMove: 移動直前の _stayTimerの値

【判定タイミング】 障害物が各マスに到達した瞬間（cellIndexが変わるタイミング）に: 到達したマスが(_lastLeftCol, _lastLeftRow)と一致し、 かつ 現在時刻 - _lastMoveTime < 0.4秒、 かつ _stayTimeAtLastMove >= 0.3秒 → ジャスト回避成立！

通常回避の判定: 障害物が到達したマスにプレイヤーがいない かつ ジャスト回避の条件を満たさない → 通常回避（コンボ+1だが低いスコア）

ラッキージャスト回避: 別の障害物を避けるために移動した結果、 たまたまジャスト回避条件を満たした場合も成立する （判定ロジック上は同じだが「LUCKY!」表示を追加） 判定方法: 移動の入力時に、移動先マスに向かってくる障害物が存在しなかった場合 つまり「この障害物を見て避けたわけではない」ケース → ジャスト回避成立 + ラッキーフラグtrue

【スコア】

- 通常回避: 100点
- ジャスト回避: 500点
- ラッキージャスト回避: 500点
- コンボ倍率: 現在のコンボ数を乗算（最低×1）
- ゾーン中: さらに×5

スコア表示:

- 画面右上にtotalScoreを常時表示（Label、フォント32px）
- 加点時にプレイヤー上に加点数をポップアップ表示: Label生成 → 上に浮遊しながらフェードアウト（0.8秒） ジャスト回避: 金色テキスト、フォント48px 通常回避: 白テキスト、フォント32px ラッキー: 虹色（とりあえず紫 Color(0.8, 0.4, 1)）、フォント48px + "LUCKY!"テキスト追加

【コンボとゾーンの更新】

- 通常回避: コンボ+1
- ジャスト回避: コンボ+2
- 被弾: コンボリセット
- 8コンボでゾーン突入（変更なし）

【HUDにスコア追加】

- 既存のComboLabelの下にScoreLabelを追加: "Score: 0"
- tscnにもScoreLabelノードを追加すること

dotnet build でエラーがないことを確認してください。 Prompt 3 / 6 — コンボ段階演出＋ゾーン演出＋被弾演出 GridSampleManager.cs にコンボ段階演出、ゾーン突入演出、被弾演出を追加してください。

【コンボ段階演出】 コンボ数に応じて画面の雰囲気が段階的に変わる:

コンボ1〜3（Nice）:

- 回避したマスが白く0.2秒光る（既存のFlashCellを流用）
- プレイヤー上に "Nice!" テキストポップアップ（白、フォント36px）

コンボ4〜5（Great）:

- マスの光が大きくなる（ColorRectを一回り大きく）
- "Great!" テキストポップアップ（水色 Color(0.3, 0.8, 1)、フォント42px）
- 画面の左右端にうっすら青いオーラを表示: 左端: ColorRect(40, 1920), Color(0.2, 0.4, 1, 0.15) 右端: ColorRect(40, 1920), Color(0.2, 0.4, 1, 0.15) Position: 左(0,0)、右(1040,0)

コンボ6〜7（Amazing）:

- "Amazing!!" テキストポップアップ（黄色 Color(1, 0.9, 0.2)、フォント48px）
- 画面の左右オーラが強く: alpha=0.25
- 画面が微振動: Tweenで全体ノードのPositionを±3pxで0.05秒×4回揺らす
- ナビ台詞: コンボ6で "すごい！この調子！"、コンボ7で "あと1回...！"

コンボ8（ゾーン突入）:

- 後述のゾーン突入演出へ

【ゾーン突入演出（8コンボ到達時）】 以下を0.3秒以内に連続実行:

1. Engine.TimeScale = 0.3 にして0.2秒間スローモーション
2. 画面全体を白くフラッシュ: ColorRect(1080×1920, Color(1,1,1,0)) → alpha=0.6 → 0.2秒で0に戻す
3. "ZONE!!!" テキストを画面中央に大きく表示（金色、フォント72px） スケール0→1.2→1.0のバウンドアニメーション（0.4秒） 1.5秒後にフェードアウト
4. Engine.TimeScale = 1.0 に戻す
5. プレイヤーModulateを金色 Color(1, 0.85, 0.2) に変更（既存）
6. グリッド全セルの枠線色を金色に変更: Color(1, 0.85, 0.2, 0.4)
7. fx_zone.webp をプレイヤー位置に表示、ゾーン中ずっと脈動（scale 0.9〜1.1ループ）

【ゾーン中の追加演出】

- 障害物がプレイヤーのいるマスを通過する際、ダメージなし＋障害物が「破砕」する演出: 障害物のTextureRectを4つの断片に分割するのは難しいので、代わりに:
  - 障害物のModulate alphaを0.3に落とす（すり抜け感）
  - 通過時にfx_hit.webpを表示＋白い破片パーティクル: 小さいColorRect(8×8, 白)を8個、ランダム方向に飛散させてフェードアウト
  - スコア加算は通常通り（×5倍率）

【ゾーン終了時】

- プレイヤーModulateを白に戻す
- グリッドセル枠線を元の色に戻す
- fx_zone.webpを非表示
- コンボリセット
- 左右オーラも非表示

【被弾演出】

- 画面全体を赤くフラッシュ: ColorRect(1080×1920, Color(1,0,0,0)) → alpha=0.3 → 0.15秒で0に戻す
- 画面を揺らす: Tweenで全体ノードPositionを±5pxで0.04秒×3回
- fx_hit.webpをプレイヤー位置に0.3秒表示
- ライフゲージの減少部分を赤く光らせてから減少: ProgressBarのfill色を赤にしてから値を減らし、0.3秒後に元の色に戻す

【ジャスト回避演出】

- Engine.TimeScale = 0.2 にして0.05秒間ヒットストップ

- プレイヤーが移動した元のマスに残像表示: 半透明(alpha=0.4)のSprite（プレイヤーと同じフレーム）を配置、0.4秒でフェードアウト ※ 残像が複雑なら、代わりにそのマスを青白く光らせる: Color(0.5, 0.7, 1, 0.5)→0.4秒でフェード
- fx_just_avoid.webpを元のマスに0.3秒表示
- Engine.TimeScale = 1.0 に戻す
- スコアポップアップ（金色、前述）

【実装上の注意】

- 演出用のColorRect/TextureRect/Labelは使い捨て（生成→Tween→QueueFree）
- Engine.TimeScaleの変更は短時間（0.05〜0.2秒）に留め、必ず1.0に戻すこと
- 複数の演出が重なる場合（ジャスト回避＋コンボ増加＋ゾーン突入）は順に実行

dotnet build でエラーがないことを確認してください。 Prompt 4 / 6 — レイアウト＋背景＋ナビゲーター＋イベント＋操作ヒント GridSampleManager.cs と GridSample.tscn のレイアウトを本番品質に更新してください。

【画面レイアウト（上から順）】

■ HUDエリア（Y: 0〜180） 左上: PhaseLabel "青年期" + AgeLabel "20歳"（フォント28px） 中央: LifeBar（幅400、高さ30、画面中央寄せ） StyleBoxFlat: 背景 Color(0.15,0.15,0.2,0.8) corner_radius=4 Fill Color(0.2,0.8,0.3,1) corner_radius=4 ライフ50%以下: 黄色 Color(0.9,0.8,0.2,1) ライフ25%以下: 赤色 Color(0.9,0.2,0.2,1) 右上: WaveLabel + ComboLabel + ScoreLabel（フォント28px）

■ ナビゲーターエリア（Y: 180〜420） 左: navigator.png（80×80に縮小） 右: 吹き出し背景（navi_bubble_thought.webpをTextureRect、240幅） 中にNavigatorLabel（フォント22px） 吹き出しフェード表示/非表示: Speak(text, duration=3f): フェードイン0.2秒、duration後フェードアウト0.3秒 発話タイミング: 開始前: "画面の下をスワイプして動こう！" Wave開始: "Wave X スタート！" 被弾: ランダム("痛い！", "気をつけて！", "まだいける！") ジャスト回避: ランダム("ナイス！", "かっこいい！", "見事！") ラッキー回避: "ラッキー！" コンボ6: "すごい！この調子！" コンボ7: "あと1回...！" ゾーン: "ゾーン突入！無敵だ！" 全クリア: "やった！全Wave突破！" 死亡: "お疲れ様...また次の人生で。"

■ グリッドエリア（Y: 440〜1340） マスサイズ: 280×280、マス間余白: 20 グリッド左上起点: X=100, Y=450 各マス中心座標（更新）: row0: (240,590) (540,590) (840,590) row1: (240,890) (540,890) (840,890) row2: (240,1190) (540,1190) (840,1190) プレイヤー初期位置: col=1, row=1（540,890） マスの見た目: Color(0.25, 0.25, 0.35, 0.3) プレイヤー在: Color(0.27, 0.53, 1, 0.25)

■ 境界（Y: 1340〜1390） 半透明ColorRect(1080×50, Color(0.1,0.1,0.15,0.3))

■ 操作エリア（Y: 1390〜1920） 背景色: Color(0.2, 0.2, 0.28, 1) 初回のみ矢印ヒント表示（操作エリア中央540,1650）: 上下左右の三角形Polygon2D、Color(1,1,1,0.25) Tweenでscale (0.9→1.1→0.9) ループ、1.2秒周期 最初の移動成功でフェードアウト(0.5秒)→QueueFree イベント時はここにイベントUIを表示

■ 背景（全画面、Z-index=-10） SkyBg: sky_bg2.webp（TextureRect、1080×1920、stretch） MidGround: midground_town.webp（画面下寄り） ※ 背景はグリッドの後ろ、HUDの後ろに配置

【入力制限】 入力開始位置がY < 440（HUD/ナビエリア）の場合はスワイプ/ガードを無視

【Wave間イベント（簡易版）】 Wave1,2クリア後の2秒待ちの間にイベント表示（操作エリアに）: 背景: ui_event_frame.webpをTextureRect テキスト+選択肢ボタン2つ（ui_event_button.webp背景のButton、各幅400×80）

仮イベント3パターン:

1. "道端で困っている人を見かけた" A: "助ける" → ナビ「優しいね！」ライフ+5 B: "通り過ぎる" → ナビ「まぁ仕方ないか」
2. "落とし物を拾った" A: "届ける" → ナビ「いいことした！」コンボ+2 B: "自分のものにする" → ナビ「うーん...」ライフ+10
3. "友人から誘いの連絡が来た" A: "遊びに行く" → ナビ「リフレッシュ！」ライフ+3 B: "断って休む" → ナビ「休息も大事」ライフ+5

選択後1秒でイベントUI非表示→次Wave開始

【死亡時演出】 画面フェードオーバーレイ: Color(0,0,0,0)→alpha=0.6、2秒 ナビ: "お疲れ様...また次の人生で。" StateLabel: "DEAD - Tap to restart"

【全クリア演出】 ナビ: "やった！全Wave突破！" StateLabel: "ALL WAVES CLEAR!"（金色 Color(1,0.85,0.2)）

【tscn更新】 レイアウト変更に合わせてGridSample.tscnを再生成。 ノード構成: GridSample (Node2D) ← script ├── Background (Node2D) z_index=-10 │ ├── SkyBg (TextureRect) ← sky_bg2.webp │ └── MidGround (TextureRect) ← midground_town.webp ├── GridContainer (Node2D) │ └── Cell_0_0〜Cell_2_2 (ColorRect) ← サイズ280×280、新座標 ├── Player (AnimatedSprite2D) position=(540,890), scale=(0.59,0.59) ├── Obstacles (Node2D) ├── WarningLines (Node2D) ├── Effects (Node2D) ← 演出用パーティクルの親 ├── UI (CanvasLayer) │ ├── HUD (VBoxContainer) ← 位置更新 │ │ ├── PhaseLabel, AgeLabel, WaveLabel, LifeBar, ComboLabel, ScoreLabel │ ├── NavigatorArea (HBoxContainer) ← Y=180 │ │ ├── NaviIcon (TextureRect) ← navigator.png │ │ └── NaviBubble (TextureRect) ← navi_bubble_thought.webp │ │ └── NavigatorLabel (Label) │ ├── StateLabel (Label) ← 位置更新 │ ├── EventPanel (PanelContainer) visible=false ← Y=1400付近 │ │ ├── EventText (Label) │ │ ├── ChoiceA (Button) │ │ └── ChoiceB (Button) │ ├── FlashOverlay (ColorRect) 1080×1920 Color(1,1,1,0) ← 白フラッシュ用 │ ├── HitFlashOverlay (ColorRect) 1080×1920 Color(1,0,0,0) ← 赤フラッシュ用 │ └── DeathOverlay (ColorRect) 1080×1920 Color(0,0,0,0) ← 死亡フェード用 └── OperationArea (ColorRect) ← Y=1390、1080×530、操作エリア背景色 └── Boundary (ColorRect) ← Y=1340、1080×50、境界

C#側のGetNodeパスとtscnのノード名を完全に一致させること。

dotnet build でエラーがないことを確認してください。 Prompt 5 / 6 — 統合テスト＋バグ修正 ここまでの実装を統合テストしてください。

【チェックリスト】

1. dotnet build が成功すること
2. 以下のGodot CLIでシーンが起動すること: & "D:\copilot_script\Godot_v4.6.1-stable_mono_win64\Godot_v4.6.1-stable_mono_win64_console.exe" --path "D:\code_workspace\generation-roguelite\generation-roguelite" --run-scene "res://Scenes/Sample/GridSample.tscn"
3. コンソール出力にエラーがないこと
4. エラーがあれば修正して再テスト

【よくあるエラーと対処】

- "Node not found" → C#のGetNodeパスとtscnのノード名不一致を修正
- "Cannot call method on null" → ノード取得失敗、tscnのノード構成を確認
- SpriteFrames関連 → AtlasTexture.Region の計算確認(col*341, row*341, 341, 341)
- tscnパースエラー → load_steps数、ext_resource id、parentパスを確認
- レイアウトずれ → ColorRectのoffset_left/top/right/bottom再計算
- 演出のTweenエラー → null参照チェック追加、QueueFree済みノードへのアクセス防止

【修正時の注意】

- 1つ修正するたびに dotnet build で確認
- 大きな変更は避け、エラーを1つずつ潰す Prompt 6 / 6 — APKビルド＋実機インストール 実機テスト用にAPKをビルドしてインストールしてください。

Step 1: project.godotのメインシーンを一時変更 run/main_scene="res://Scenes/Main.tscn" → run/main_scene="res://Scenes/Sample/GridSample.tscn" BOMなしUTF-8。他の行は変更しない。

Step 2: dotnet build dotnet build D:\code_workspace\generation-roguelite\generation-roguelite\GenerationRoguelite.csproj

Step 3: adb接続確認 & "C:\Users\mikami\AppData\Local\Android\Sdk\platform-tools\adb.exe" devices

Step 4: エクスポート＋インストール cd D:\code_workspace\generation-roguelite\generation-roguelite & .\tools\export_and_install_android.ps1 -GodotExe "D:\copilot_script\Godot_v4.6.1-stable_mono_win64\Godot_v4.6.1-stable_mono_win64_console.exe" -LaunchAfterInstall

失敗時フォールバック: a) $env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"; $env:GRADLE_JAVA_HOME = $env:JAVA_HOME b) & "D:\copilot_script\Godot_v4.6.1-stable_mono_win64\Godot_v4.6.1-stable_mono_win64_console.exe" --headless --path "D:\code_workspace\generation-roguelite\generation-roguelite" --export-debug "Android" "build/android/generation-roguelite.apk" c) & .\tools\sign_and_install.ps1 -LaunchAfterInstall

Step 5: project.godotを元に戻す run/main_scene="res://Scenes/Sample/GridSample.tscn" → run/main_scene="res://Scenes/Main.tscn" BOMなしUTF-8。

エラー対処:

- "Unsupported class file major version" → JAVA_HOME をJDK 17/21に
- "INSTALL_FAILED_UPDATE_INCOMPATIBLE" → adb uninstall com.example.generationroguelite
- Gradleエラー → Remove-Item -Recurse -Force ".gradle-export\daemon"

Step 5は必ず実行すること。 実行順序 事前準備プロンプトを渡す Prompt 1 → 障害物を軌道貫通型に全面書き直し Prompt 2 → ジャスト回避＋スコアシステム Prompt 3 → コンボ段階演出＋ゾーン＋被弾演出 Prompt 4 → レイアウト＋背景＋ナビ＋イベント＋操作ヒント Prompt 5 → 統合テスト＋バグ修正 Prompt 6 → APKビルド＋実機テスト